IDRN Global Connections Technical Description

Technical review draft prepared for Rob Phillips

Prepared at the request of Matt Wallace for technical review by Rob Phillips of Grokism.io. This document describes IDRN's organization, implementation, recorded design decisions and outstanding work. It does not assess Grokism, choose an external identity provider or propose an integration architecture.

Document date: October 23, 2026, America/Chicago. Version: Technical draft 1.0. Source baseline: 2fad378739aaaedeb76f601228f8bcfa268dafe0 in jmattwallace/idrn-registry. This technical handoff contains no credentials, invitation tokens, private account listings or registry exports.

The principal finding is that IDRN has several distinct security and data paths. The main registry retains a full-graph browser client and broad authorized-user database access. Events uses an isolated administrator-only server API. Invitations and contributions use separate functions and email-token workflows. More restrictive registry access code exists locally but is not enabled. A protection present in one subsystem cannot be assumed to cover all the others.

Contents

1 Evidence and status conventions 4
2 Purpose and functional organization 4
3 Runtime architecture and technologies 5
4 Source organization and source authority 5
5 Registry graph model 6
5.1 Nodes and identifiers 6
5.2 Relationships and attribution 7
5.3 Taxonomy and reconciliation 7
6 Persistence caching and data transfer 7
7 Accounts authentication and person identity 8
7.1 Separate records and meanings 8
7.2 Sign in and session restoration 8
7.3 Current matching and limitations 9
7.4 Roles and designators 9
8 Authorization privacy and enforcement gaps 10
8.1 Current database boundary 10
8.2 Browser controls and visibility 10
8.3 Staged secure route 10
9 Invitations contributions and membership 11
9.1 Released email confirmed review 11
9.2 Source authority and tested result 11
9.3 Other intake and onboarding 11
10 Events administration 12
10.1 Separate application and production codebase 12
10.2 Current access checks 12
10.3 Data model and record links 12
10.4 Categories calendar and time 13
10.5 Consistency and initial limits 13
11 Deployment publishing and operational boundaries 14
11.1 Current release path 14
11.2 Local push button and ordinary Git 14
11.3 Why whole repository publication was replaced 14
11.4 Backend releases and rollback 15
12 Monitoring and test evidence 15

12.1 Aggregate health reporting 15
12.2 What the available tests establish 15
13 Recorded design decisions and why they matter 16
14 Owner approved policies not yet certified as implemented 17
14 1 Standard user access 17
14 2 Separate full visibility permission 17
14 3 Permanent account to person association 17
14 4 Attribution does not confer editing rights 17
14 5 Member editable and protected fields 17
14 6 Separate contact and login email 18
14 7 Sensitive cache lifecycle and transfer frugality 18
14 8 Meaning of Open visibility 18
14 9 Separate verification claims and provenance 18
14 10 Verification history and permission review 18
14 11 Scoped verification reviewers 19
14 12 Verification does not grant access 19
14 13 Relationship assessment correction review 19
14 14 Account retirement preserves person history 19
15 Remaining work and unverified areas 19
15.1 Reconcile authorization enforcement 19
15.2 Resolve identity and account continuity 20
15.3 Balance caches and transfer cost 20
15.4 Complete lifecycle and operational evidence 20
16 Verification boundaries for future document revisions 20
17 Recorded release timeline 21
18 Glossary 21
19 Source reference index 22
Release links 24
20 Preserved deployed function inventory 24

1 Evidence and status conventions

Implemented and release verified means inspected source plus preserved release or acceptance evidence support the statement for a named release. **Implemented in source** means code inspection supports the behavior, without implying a new live production test. **Staged or preserved** means implementation/design exists but is not enabled in the current production route. **Approved policy** means the owner confirmed intended behavior during the October 23 walkthrough; approval is not evidence of implementation. **Remaining work** identifies an enforcement, compatibility or verification gap.

Evidence comprises source inspection, deployment manifests, receipts, recorded tests and owner-confirmed decisions. This report is not a fresh audit of all production accounts, data, permissions or mail deliveries. Older notes sometimes say local-only or disabled because later releases superseded them. Current completion records and exact source authority take precedence. [S01, S02, S15–S18]

The website baseline is PR #7's merge. The preserved backend inventory verified at 2026-10-23T05:03:24.244Z showed 45 functions: 43 preexisting functions and two new Events functions. That is approximately 00:03 Chicago time on October 23. The rules were unchanged during the Events deployment. Subsequent Events form fixes were website-only. These are dated records, not continuously refreshed observations. [S16, S18, E01]

No application code, account, rule, runtime flag or production record was changed for this report. The owner's new policies are recorded separately from executable source. This document's assembly does not implement those policies or authorize a production release.

2 Purpose and functional organization

IDRN Global Connections is a relational network registry for preparedness, disaster response and practical collaboration. It models people, organizations, geography and service sectors as connected entities. The map supports discovery and understanding of relationships. Recorded principles emphasize thoughtful introductions, consent, privacy, reciprocal contribution and follow-through. [S03, S04]

The main experience provides a force-directed graph, search, focused neighborhoods, connection-path exploration, entity details and relationship editing. Administrative tools support data entry, imports, duplicate reconciliation, accounts, feedback, backups and publishing. Events is a separate administrative application launched from the registry; it does not replace the master graph interface. [S03, S05, S16]

A person in the network is distinct from a person with an authorized login. Prospects, historical contacts and event contacts can exist without login accounts or membership. A login can have an application authorization profile, an audit author and a linked registry person. Those are related records with different responsibilities.

Surface	Responsibility	Current technical boundary
Main registry at reg-idrn.info	Graph, discovery, profiles, relationships and administration	Static browser application with Firebase Auth and legacy direct database access
invite.html	Invitation request, editable review and confirmation	Dedicated client and isolated email-token functions
contribute.html and pledge-intake.html	Contribution and intake workflows	Dedicated clients, callable services and scoped token/session handling
idrn.info/register	Membership acceptance and help	Separately hosted assets and function releases; outside the registry publisher catalog

Surface	Responsibility	Current technical boundary
events-admin.html	Events, crises, participation and follow-up	Administrator-only server API and private Events records
Local development server	Preview and development administration	Local tool, including code publishing; not the production web server

3 Runtime architecture and technologies

The website is principally static HTML, CSS and JavaScript. src/app.js is the main controller; ES modules provide storage, taxonomy, country references and workflow clients. D3 version 7 renders the network. A vendored spreadsheet library supports spreadsheet-oriented tools. Hostinger serves the UI; server.js is a local development/administrative helper rather than a required production Node web server. [S03, S05, S19]

The inspected client imports Firebase Web SDK 10.12.4 modules from Google's gstatic CDN. It uses Firebase Authentication, Realtime Database and callable Functions. The project is idrn-registry with its default Realtime Database instance. This is a JSON tree and graph-record design, not Cloud Firestore documents or SQL tables. [S06]

Privileged functions use the Firebase Admin SDK. The preserved deployment inventory contains both Node.js 20 and Node.js 22 runtimes. Root functions/package.json declares Node 20 and is not the runtime authority for all deployed functions. The reviewed invitation loader/confirmation and Events packages use Node 22. A wholesale root backend deployment would be substantially broader than one selected function release. [S15, S16, E01]

Cloudflare caching and observed HTML transformations are accounted for in public verification. SMTP2GO supplies transactional email. Firebase functions support account operations, intake, invitations, contributions, private assessment endpoints and aggregate monitoring. Separately hosted membership pages have a separate publication lifecycle. [S15, S19, S20]

```
Main registry browser
  -> Firebase Authentication -> authorized_users profile
  -> direct Realtime Database access -> full graph and legacy local cache

Events browser
  -> same-origin Firebase login -> ID token -> eventsAdminApi
  -> current account and admin checks -> private Events storage

Invitation browser
  -> request callable -> review email -> scoped token
  -> load and confirm -> corrected frozen list -> email provider

Reviewed source -> GitHub main -> website-only Actions publisher
  -> pinned SSH and SFTP destination -> origin and public verification
```

Static hosting does not provide IDRN account authorization. A visible administrator button is not an authorization boundary. Likewise, a publicly invokable function can still require application-level authentication or a valid workflow token. These distinctions explain why each path needs its own evidence.

4 Source organization and source authority

The current daily-development checkout is global-connections-github. The older global-connections candidate and original source directories are preserved for unreleased work and history. They must not be copied or

merged wholesale into the current release. Private snapshots and preservation records are outside the website payload. [S01, S02]

Location	Responsibility	Qualification
index.html and styles.css	Main shell and shared appearance	Changes may affect many workflows
src/app.js and src/storage.js	Interface, presentation policy, graph state and persistence	Contains active legacy and opt-in secure paths
src/registryAccessClient.js and registryEditOperations.js	Server views and deliberate revision-based patches	Staged registry route
functions/registrySecurity.js and registryAccess.js	Actor resolution, filtered views and checked commits	Access handlers are not the active exported route
release-prep/invitation-review-loader and invitation-review-recovery	Archived exact deployed invitation sources	Different selected service versions; do not mix them
src/events and functions/events	Separate Events interface, model, schedule and services	Local demo code coexists with production adapters
functions/events-production and release-prep/events-production	Production wrapper and locked package inputs	Explicit two-function idrn-events codebase
security	Candidate rules, emulator fixtures and host protection	A file's presence does not establish deployment
scripts and docs	Tests, publisher, release records and decisions	Excluded from website uploads

Comments can lag releases. The reusable Events service still has an unreleased comment, but its production wrapper and release evidence establish deployment. Conversely, shipping the registry-access adapter as a static file does not enable its flag or export its backend handlers. Route, configuration, package and completion record must be read together. [S07, S16]

5 Registry graph model

The idrn_registry root stores nodes, edges and update metadata. Helpers normalize arrays and keyed representations encountered during Firebase round trips. D3 can enrich endpoint references and add layout coordinates. Reconciliation comparisons exclude layout-only values while retaining meaningful record content. [S03, S06, S09]

5.1 Nodes and identifiers

Nodes represent people, organizations, locations and service sectors. Common fields include id, label and nodeType. Other fields cover contact channels, websites, images, taxonomy, organization type, visibility, aliases, membership status and authorship. Geography includes country/continent hierarchy and reference links. [S03, S05, S10]

Person records contain several different identities: ordinary contact email, account-related email fields, person_status/designator and membership state. Access profiles may link explicitly through person_node_id.

These fields developed through different workflows. A historical field named verified_account_email does not prove every stored value has identical provenance or that Firebase Authentication currently reports emailVerified=true. [S08, S11]

IDs are referenced throughout relationships, accounts, invitations and Events. Some historical IDs derive from names and reveal identity. The staged server-view model replaces other people's canonical IDs with viewer-specific opaque references. Permanent person-ID login linking is now approved policy, but it is not consistently enforced by every active legacy path. [S07, S08]

5.2 Relationships and attribution

Typical relationship fields include id, source, target, type/relationshipType, strength, subject_id, author_id, timestamp and note. Newer and historical records also use created_by, edited_by and their timestamps. Source/target helpers accept either IDs or D3-enriched endpoint objects. Administrative audit relationships are distinct from networking edges. [S03, S05, S09]

Primary categories are Personal, Organizational, Geographical and Service Sector. Endpoint direction and category affect matching and deduplication. CSV imports can update a matching relationship rather than duplicate it. The interface includes historical strengths from -4 to +4. These are relational assessments, not authentication credentials or evidence of verified identity. [S03, S05]

Relationship point limits keep recorded networks focused. The documented default is 100, with warning near 85 percent and confirmation treatment for imported overages. These UI/model limits differ from database enforcement. The recorded rules allow newly written shared strengths from 0 through 4, or an unchanged existing value. [S03, S12]

The current store sends newly changed negative strengths through savePrivateRelationshipAssessment and prevents those new negative values from entering the shared graph. Both private-assessment functions appear in the preserved deployed inventory. This confirms source routing and function presence, not a fresh audit of their independently deployed implementation or all historical negative data. [S06, E01]

5.3 Taxonomy and reconciliation

IDRN maintains service-sector and geography taxonomy. Country-to-continent links support navigation. Organization types include government, business, network, faith-based, civic and other. Seed/reference normalization can reconcile older data, which is one reason the secure opt-in route suppresses automatic seed insertion and name-based reconciliation. [S03, S10, S11]

Duplicate merging rewrites references to a surviving node, can retain aliases and creates audit history. Merging affects identity continuity and differs from renaming a label, changing contact email or retiring a login. No merge is authorized or performed by this documentation work.

6 Persistence caching and data transfer

Implemented and release verified: PR #2 added one root listener per RegistryStore, explicit teardown, generation guards and recovery after an initially denied subscription. It preserves full graph/search/connection-path behavior. It does not implement paged graph access or establish a measured production billing reduction. [S06, S13]

In the active legacy route, the store can load a full graph from localStorage and merge seed data, then subscribe to idrn_registry with onValue. Incoming snapshots are normalized, cached and delivered to the application. The persistent graph key is idrn-network-registry-v1. Preferences and application sessions use other keys. Firebase sign-in persistence is independently set to browserLocalPersistence. [S06]

Live saves use a root transaction when available, preserve unrelated remote records and verify required records after confirmation. Metadata marks live edits and guards against a development snapshot overwriting known live changes without reconciliation. Fallback paths perform root updates. This remains broad graph persistence rather than a fully server-authorized patch API. [S06]

Explicitly offline remote saves are rejected before optimistic caching, with the record-editor draft retained. An online browser indication is only a hint; server confirmation is necessary. There is no general automatic offline write-replay promise. Localhost development saves have separate local behavior. [S13]

The full-app acceptance record observed two transactions during an exercised editor save: one deliberate edit and one preexisting normalization save, with one audit addition. That behavior was preserved. The release did not prove one transaction per edit, a shared listener across tabs/users or an 80 percent download saving. [S13]

Staged or preserved: a compact graph overview/search/detail prototype and account-scoped memory client remain outside the active app route. The accepted direction preserves complete authorized connection paths while reducing unnecessary detail transfer. Legacy persistent full-graph caching has not been replaced. The owner's new security/frugality policy still requires concrete sensitivity, cache-validity and revalidation rules. [S02, S13]

7 Accounts authentication and person identity

7.1 Separate records and meanings

Concept	Representation	Meaning
Login	Firebase Authentication UID and user account	Firebase sign-in identity and token subject
Application access	authorized_users/{uid}	IDRN access profile and role
Audit author	author_id and author_name	Application attribution and historical authorship
Registry person	Person node ID	Network entity referenced by accounts, relationships and Events
Account association	person_node_id/node_id and legacy matching	Connects an account to a person; behavior differs by route
Membership and engagement	person_status/designator and membership_state	Network classification, separate from admin role
Verification	Email fields and individual workflow evidence	No single uniform identity/authority claim model is established throughout the current system

A registry person does not imply a login. A Firebase login does not by itself imply an authorized_users entry. An access profile does not establish that all legacy matching finds the correct person. A role, engagement level and verified claim have distinct meanings. [S05–S08, S11]

7.2 Sign in and session restoration

The main production interface signs in with Firebase email/password through signInWithEmailAndPassword. It then reads authorized_users/{uid}. If no profile exists, the login handler signs out and reports that the account is

not authorized. Recognized profiles become browser application sessions with author identity, role, email and Firebase UID; an is_admin flag supports interface behavior. [S05, S06]

Backend functions support account creation, access invitations, login recording and password-reset completion. The interface handles accounts required to replace a temporary invitation password before continuing. Firebase persistence restores login across reloads; application session information is also in localStorage. These browser values are not independent server-issued permissions. [S05, S06, E01]

Local development has a separate session/account path. A local administrator flag can reveal prototype navigation. Production Events instead requires a real Firebase identity token and server-side administrator approval. Local flags cannot authorize its server API. [S16, S17]

7.3 Current matching and limitations

The active legacy browser function isSessionPersonNode can match normalized author/node identifiers, normalized labels or contact email. personNodeForAccessUser also contains creator/restored-author matching. These inspected paths do not satisfy the newly approved permanent person-ID rule. [S05]

The staged registry server policy is stricter. registryActor first resolves explicit person_node_id/node_id, then exact identifier matching, and finally a constrained protected-account-email fallback. Ambiguous or unlinked accounts cannot claim a person under that policy. Because fallback matching remains, staged code also needs reconciliation with the newly approved permanent-ID requirement. [S08]

Recorded October account maintenance added reviewed explicit links and protected email fields while retaining roles, verification flags and historical references. Unresolved shared-person cases were retained rather than guessed. Earlier findings included unverified Firebase email flags even when account/profile email values agreed. Those dated findings are not a new live account audit. [S11]

Stricter requester lookup also exists in root backend source. Its presence does not prove every separately deployed legacy function has those same bytes. Per-function source authority must be checked before describing an independently deployed endpoint's exact ownership resolution. [S11, S15]

7.4 Roles and designators

Application access roles are admin and user. The interface labels them System admin and Standard user. A designator describes network involvement; it is a different field from role. [S05]

Level	Current interface label
0	Unaware
1	Friend of IDRN
2	Registered
3	Pledged
4	Active
5	Expert
6	World-Class Expert
7	Service Sector Mgr
8	New Launch Key
9	Legacy

The current code grants Level 9 full display visibility without granting administrator tools. That coupling is an existing implementation decision, now superseded as intended policy by the owner's requirement for a separate full-visibility permission. It has not been removed. Membership progression, organizational representation and identity verification must not be inferred from this level table alone.

8 Authorization privacy and enforcement gaps

8.1 Current database boundary

The rule-only baseline captured for the Events release denies access at the root but grants read and write on idrn_registry and idrn_registry_meta when an authenticated UID has an authorized_users entry. The grant tests profile existence, not role, active state, field privacy, ownership or the new policy decisions. Other protected trees inherit root denial unless specifically allowed. [S12, E01]

The same baseline lets a user read their own authorized_users profile and administrators read other profiles. Profile writes require an administrator profile. Feedback has its own conditions. Those narrower rules do not narrow the whole-registry grant. UI hiding and disabled buttons therefore cannot be treated as enforced database-level privacy. The graph is not anonymously public, but authorized-user access is broader than the new intended policy. [S12]

8.2 Browser controls and visibility

The legacy interface allows administrators to edit broadly and standard users to edit creator-owned or self-matched nodes. Edge editing generally tests creator/author attribution. Contact display uses role, self-editability and visibility policy. The adopted-user view lets an administrator preview another user's interface while the real administrator remains signed in. A preview is not a database isolation test. [S05]

hasFullRegistryVisibility grants full display visibility to administrators or designator 9. The staged registryActor uses a corresponding fullRead condition. Both need reconciliation with the newly approved separate permission; neither should be reported as already corrected. [S05, S08]

The interface offers Open, Normal, Limited and Private plus field-level preferences. The owner has clarified that Open means authorized, signed-in IDRN users. Internet publication requires separate explicit choice. Broad authenticated graph reads remain an enforcement gap even where UI privacy labels and aliases display correctly. [S05, S12]

8.3 Staged secure route

IDRN_REGISTRY_ACCESS_MODE='server' is the opt-in switch. It is not enabled in the checked-in production configuration. In that route RegistryStore loads filtered server views, clears secure state on authentication changes, avoids a local full-graph fallback and submits deliberate operations with base revisions. RegistryAccessClient rejects stale identity/view responses and does not blindly replay uncertain commits. [S06, S07, S11]

getRegistryView and commitRegistryEdits use trusted Firebase identity/profile information. Policy code filters records/fields, assigns editability, derives viewer-specific HMAC references and applies revision-checked field patches with server-attributed audit records. The access handlers are deliberately not exported as the active root route. Candidate rules are not live rules. [S07, S08, S11]

Isolated tests support this candidate, but complete editor/admin compatibility and all older callable surfaces remain release gates. Candidate rules retain administrator root access for legacy bulk tools. This is not an

already deployed, complete server-only boundary. The new owner policies add further requirements, including removing automatic creator edit grants and Level 9 full-read grants.

9 Invitations contributions and membership

9.1 Released email confirmed review

The public invitation page accepts an initiating member email and one to six proposed people. It returns a generic response and emails a private review link. For this workflow eligibility requires a valid unexpired token and an unambiguous current registry-person email match. A Firebase login is not required, and the released review path does not add a membership-approval/designator gate. Older signed-in invitation paths and design notes have different thresholds; they must not be generalized to this route. [S14, S15]

The archived confirmation service generates 32-byte random tokens and stores SHA-256 token identifiers. Review links use a URL fragment and expire after 24 hours; recipient invitation/connection tokens use a 30-day window. Private state includes batches, requests, outcomes, connections, pending entries, deliveries, limits and audit under member_invitation_requests. Token validity authorizes the scoped workflow, not general registry access. [S14, S15]

The released review page permits editing names and addresses before confirmation. The confirmation service accepts revisedPeople, validates and normalizes it and transactionally freezes the corrected list before sending. The loader exposes processing/locked states and permits status refresh without sending. Cold-cache recovery preserves corrections when another tab or earlier request has confirmed the same list. [S15]

A new address can receive a membership invitation before a person node exists. An existing person can receive a consent-based Personal connection request. Already-connected people receive no new connection request. For an existing member, delivery uses the corrected email while the greeting can use the registered name. [S14, S15]

Batch, row and delivery guards protect repeat confirmation of the same list. An intentional new list can permit a resend. Failed or uncertain delivery is not automatically replayed. Provider acceptance, inbox receipt, consent, registration and graph relationship creation are separate states. [S14, S15]

9.2 Source authority and tested result

loadMemberInvitationBatch is archived in release-prep/invitation-review-loader; confirmMemberInvitationBatch is archived in invitation-review-recovery. They preserve different selected service versions and pinned dependencies. The root functions/index.js generic wrapper passes only a token on its confirmation route, so it is not the source authority for the tested revised-people behavior. Deploying the root package wholesale risks regressing it. [S15]

The October 20 controlled inbox test was owner-confirmed: one review and one corrected invitation arrived; recipient and greeting matched the corrections; the original address received nothing; and reopening the review retained corrected locked details. Addresses and token links are excluded here. This minimal production test did not prove every race, failure or registration path; those have separate isolated checks.

9.3 Other intake and onboarding

The inventory includes public intake, contribution-session, acceptance, account and membership functions. Registration assets at idrn.info/register have their own release lifecycle. Workflow state and report queues are handled through dedicated functions and scoped tokens/sessions. Source contains private defaults, country/language fields, pledge progression and quiet-period reporting logic. [S11, S14, S21, E01]

Older onboarding/gathering documents explicitly label some work as local or proposed. Their presence does not prove production release of every redesign. Language-aware gatherings, RSVPs, orientation, capacity, badges and recordings must not be conflated with the current administrative Events application. Detailed live claims about legacy acceptance require each deployed function and separately hosted page's own evidence. No new end-to-end registration test was performed for this report.

10 Events administration

10.1 Separate application and production codebase

Implemented and release verified: PR #5 released Events as a separate administrative page with real shared storage and registry-host lookups. The main interface received its launcher and scoped styling. Synthetic fixture/demo adapters remain test tools and are excluded from the public runtime catalog. PR #6 corrected Create-control visibility; PR #7 clarified organization links and removed synthetic wording from production forms. [S16–S18]

The isolated idrn-events codebase exposes eventsAdminApi and projectEventsSummary in us-central1 on Node.js 22. The API has 256 MiB, a 60-second timeout, maxInstances=3 and concurrency=20. These configured bounds are not load-test throughput claims. The summary trigger retries safe repair operations. Events sends no email and creates no memberships. [S16, S22]

10.2 Current access checks

The production client enables its connection only at https://reg-idrn.info. It initializes/reuses Firebase Auth with browserLocalPersistence, waits for readiness, requires a current user and supplies that user's ID token. Account-generation checks reject stale responses. A local admin flag cannot authorize this API. [S17]

The server verifies the ID token, loads the current Authentication user, rejects disabled users or auth_time older than tokensValidAfterTime, and reads authorized_users/{uid}. Its exact profile condition requires role='admin' and disabled not true. This must not be generalized into a claim that all subsystems check every active/reset/profile flag identically. There is another actor check before event mutation. [S22]

The HTTP adapter permits the production browser origin and sends no-store responses. CORS supplements token checks; it does not replace them. GET routes cover /session, /events, /hosts, /references and /events/{id}; POST /mutate performs writes. Failures map to appropriate 401/403/404/409/400/429 responses; internal failures are generic. [S22]

Direct browser access to Events database trees is denied in the captured rules; authorized service code uses Admin SDK. The HTTP handler has an in-memory limit of 120 requests per authenticated user per minute per instance, not a globally coordinated quota. Content type, size, identifiers, nesting and category fields are validated. [S12, S22]

10.3 Data model and record links

Record	Current responsibility
idrn_event_records/{eventId}	Authoritative aggregate: event, contacts, participants, organizationLinks, followUps, audit and operation fingerprints
idrn_event_summaries/{eventId}	Derived calendar/list fields; not the authoritative aggregate
idrn_event_crises/{crisisId}	Crisis background shared by multiple Response Efforts

Record	Current responsibility
Registry participant	Existing person ID, event role, attendance, evidence and optionally represented organization
Event contact	Event-scoped nonmember; no automatic registry person, login or membership
Organization link	Organization's event role such as participant, organizer or partner
Follow up	Task, responsible registry person, due date and status

Organization This Person Represents at This Event describes a person's representation for that event. Organization Linked Directly to This Event describes the organization's own participation and requires no named individual. Both can coexist. Neither changes general organization relationships in the graph. Recording participation does not automatically change connection scores or create networking edges. [S18, S23]

10.4 Categories calendar and time

Categories are Conference, Meet Up, Meeting, Training and Response Effort. Each has specific inputs. Meet Ups are structured and host-led; online/hybrid forms offer WhatsApp, Signal, Telegram, Zoom, Google Meet and Other plus sectors/geography. Response Effort references a crisis and records status, history, area, needs, objectives and coordination. Meeting, Training and Conference retain agenda, learning or organizer fields. [S23]

Scheduling stores local dates/times and IANA time zone, with calculated UTC display. Choices use browser-supported zones plus helper handling of UTC and valid current selections; this is not a guarantee of every obsolete alias on every engine. A known start time is required unless Time Not Yet Known is selected. Nonexistent/ambiguous daylight-saving times are checked, with earlier/later occurrence choices where needed. Calendar display has its own selected zone; event details retain original zone and UTC context. [S23]

Calendar/agenda share category colors and labels. Create Event/Create Crisis and Cancel stay outside independently scrolling fields. Existing records use Save. Desktop/mobile checks exercised all categories and multiple viewports with intercepted identity/API fixtures; they are not a new live data-creation test. [S18]

10.5 Consistency and initial limits

Per-event transactions use expectedRevision and stable operation IDs. The service fingerprints action/data/revision and rejects reusing an operation ID for different content. The browser retains the ID for the same uncertain save so a deliberate retry can check or finish it. This idempotent Events behavior must not be generalized to email sends. [S17, S22]

Audit and record changes commit within the aggregate. Summary projection follows that commit. A revision-guarded trigger repairs interruptions without allowing older summaries to overwrite newer ones. Recent operation fingerprints are bounded, so this is not an unlimited permanent idempotency ledger. Aggregate size is capped at 750,000 bytes and requests at approximately 45,000 bytes. [S22]

The registry directory caches IDs/names/types privately within a function instance for 60 seconds. It refreshes for save validation and returns 20-record pages. It still reads registry nodes to build its projection. Production list paging reads the small summary catalog before sorting/paging. Both have a 20,000-record guard; crisis reference choices cap at 200. No indexed large-scale query savings are claimed. [S16, S24]

Text search covers loaded summaries/current detail, as the interface states. Load More extends the set; paging does not make the search global. A host-index helper exists but is not the selected production directory path.

Archive/restore retains records; no ordinary delete operation exists. Invitations, reservations, reminders, video-platform meeting creation and recording are outside the released Events scope.

11 Deployment publishing and operational boundaries

11.1 Current release path

Implemented and release verified: the corrected GitHub Actions publisher is active for main. Hostinger's independent GitHub App auto-deployment remains owner-confirmed off. A main update can therefore publish a live website change; feature-branch publication is not itself a website deployment. The workflow uses explicit activation/origin gates and a main/repository check. [S01, S19]

The reviewed public catalog contains 36 exact runtime files. The publisher builds from immutable Git objects, validates dependencies and compares against the preceding known commit. Changed JavaScript/CSS requires updated selectors in referring assets. This protects against pairing new code with old cached dependencies. Files publish in dependency-aware order and the build marker is last. [S19]

The transfer uses a dedicated key and pinned SSH host identity. It checks the exact website destination, allowed paths and before/after hashes; rejects unexpected drift, symlinks and unsuitable parents; creates only named parents for approved new files; and verifies both origin bytes and selected public resource URLs. The publisher has no remote deletion operation. Secrets are not included in source, public artifacts or receipts. [S19]

This process reduces scope errors and detects mismatched files. It is not an atomic switch of the whole site: a brief mixed-file interval and preexisting CDN caches remain relevant. Receipt verification is therefore combined with resource selectors and compatibility testing. A transport failure can stop before uploads; uncertain partial results require inspecting the receipt, not assuming a clean rollback.

11.2 Local push button and ordinary Git

The local app's Push app to GitHub helper uses GitHub APIs and a privately saved, Git-ignored credential. Its code path targets main regardless of the local feature branch. Its root source allowlist does not include events-admin.html, although nested src/events paths are supported. Thus Events releases used ordinary Git branches and reviewed PRs. The push button and normal git push are different implementations, and both can ultimately trigger publication when main changes. [S01, S19]

The local development server exposes administration and references real configuration. It is not an offline-test server. Isolated browser tests instead fulfill approved requests with fixtures or use explicitly configured local Firebase emulators. The report records this separation because starting a local server is not proof that a test is disconnected from real services. [S01, S13]

11.3 Why whole repository publication was replaced

During the earlier release reconciliation, Hostinger's separate GitHub App deployed main despite GitHub Actions being disabled. A repository-wide publication placed source, release material and a historical registry backup under the public root. This explained how changes reached the live site without a matching Actions run. The owner paused that independent route and later authorized the exact website-only publisher. [S02, S19, S25]

A scoped .htaccess protection file was uploaded and checked, and five remaining cached source URLs were purged and verified blocked. Existing extra server files were retained; this was protection and cache handling, not deletion. The retained files, possible historical access and completeness of a broader exposure investigation must not be inferred from a later successful 403 check alone. This document includes no private file contents or sensitive transport details. [S19, S25]

11.4 Backend releases and rollback

Website publication does not deploy Firebase functions or rules. The invitation loader and confirmation have their own isolated source archives. Events is built from a named file set and pinned dependency lock, then deployed only as functions:idrn-events. Older root functions are not replaced. Separately hosted registration pages are another release boundary. [S15, S16]

Rollbacks must restore a compatible set of files and version selectors. Reverting a UI does not authorize deleting Events records, disabling accounts, retrying mail or replacing unrelated functions. The recorded Create-controls and form-wording releases name exact two/three-file rollback scopes. This is why release provenance is part of the architecture rather than an optional administrative detail. [S18]

12 Monitoring and test evidence

12.1 Aggregate health reporting

checkIdrnSystemHealth is a scheduled Cloud Function in the preserved inventory, not evidence of a Dot/BB automation. Source schedules hourly checks and aggregate emails daily or when alert severity changes. It reads bounded collections for invitations, delivery outcomes, connections, queued reports and feedback, plus SMTP2GO usage. Reports include an as-of time and explicit unavailable/truncated states. Missing data must not be interpreted as zero. [S20, E01]

Recorded exclusions include Firebase billing, database bandwidth/storage, function quotas/errors, SMTP2GO daily/hourly/API-key limits, all legacy delivery outcomes, and website availability/backups. The monitoring is not a complete system all-clear. This report does not assert that the scheduled job is currently healthy or that the recipient has a recent report; no fresh operational run was inspected.

12.2 What the available tests establish

Evidence	Recorded result and scope	Limit
Invitation review	Exact isolated source checks; 18 acceptance and 16 mixed-cache browser cases; paired emulator tests; controlled two-message production test	Minimal inbox result does not prove every live failure or registration path
Listener/offline release	199 offline/reference tests, 12 focused browser cases, 24 full-app actual SDK cases and six Auth/RTDB/Admin SDK emulator cases	Synthetic/demo data; no measured billed transfer saving
Initial Events release	257 full offline tests, nine production-path emulator/browser scenarios and eight local preview scenarios	Browser identity SDK shims in acceptance; no production seed or live event write for verification
Create controls fix	259 offline checks and five responsive cases; actual-live-assets checks	Intercepted identity/data; no real event submission
Latest form labels	258 initial passes; one catalog guard detected HEAD changing during the run and passed its stable-commit recheck; all 259 cases verified across those runs	Do not report this as a single uninterrupted 259-pass run

Evidence	Recorded result and scope	Limit
Latest live publication	PR #7, Actions run 39906756036, verified receipt: only admin.js and events-admin.html uploaded	Static delivery evidence, supplemented by five intercepted live-asset viewport checks

Tests distinguish pure policy/model checks, mocked transports, real emulator services, actual browser SDK usage, static live assets and owner-observed inbox behavior. They provide different evidence. No security assurance is inferred merely from a screenshot, syntax success, a hidden button or a handler invoked directly without its HTTP wrapper. No application tests were rerun solely to create this report; it cites dated release evidence. [S13, S15, S16, S18]

13 Recorded design decisions and why they matter

Decision	Recorded reason	Consequence today
Preserve the graph and relationship paths	Users need complete authorized connections and ConX paths	Listener optimization retained full-root behavior rather than truncating the graph
Separate registry people from logins	Contacts and prospects can matter before registration	Identity links must be explicit and separately governed
Email confirmation before sending invitations	A thoughtful invitation requires explicit member confirmation	Token review, corrected-list freeze and duplicate-send protections
Preserve exact deployed function packages	Root source and live functions had diverged	Function-specific source authority and narrow deployments
Keep Events separate	Owner did not want Events added to the master interface	Separate page and API, launched through one registry navigation entry
Model crisis separately from response effort	Several IDRN activities may address one disaster	Shared crisis reference plus separate response records
Keep event contacts outside membership	Attending or being met at an expo is not consent to join	No automatic membership, graph edge or email
Use named time zones and UTC together	Events span locations and daylight-saving rules	Local scheduling intent and unambiguous UTC context are both retained
Use server authorization for new Events	Private shared administration should not rely on browser flags	Administrator checks in every API route and denied direct data access
Stage the registry security migration	Existing bulk/editor behavior needs compatibility work	Hardened code exists but the legacy route remains active
Preserve uncertain delivery instead of auto resend	Network/provider ambiguity can cause duplicate mail	Admin/provider investigation before an uncertain resend
Replace broad static publication	Unintended files were exposed and two publishers overlapped	Exact catalog, one active publisher, separate protection and cache checks

The reasons above are drawn from recorded user decisions and release notes. Where the original choice was not documented, this report describes the mechanism without inventing a historical rationale. In particular, no

claim is made that the legacy database's broad access or its identity fallbacks meet today's approved security policy.

14 Owner approved policies not yet certified as implemented

All 14 decisions below were confirmed by the owner on October 23, 2026. They state intended requirements. Some existing mechanisms overlap them, but the walkthrough did not implement or comprehensively verify them. Each entry separates the approved rule from current implementation or remaining work. No role, account, privacy setting or production record changed.

14 1 Standard user access

Approved policy: Standard users may see only information permitted by privacy policies and edit only records they own or are explicitly authorized to manage.

Current status and remaining work: The captured authorized-user database grant is broader than this policy. Browser controls do not close that boundary. The staged route also needs review against the complete approved policy set.

14 2 Separate full visibility permission

Approved policy: Full registry visibility requires a separate explicitly granted permission; Level 9 Legacy status alone must not grant full visibility.

Current status and remaining work: Current browser and staged server logic still use Level 9 for full visibility. A separate permission model and its grant/revocation behavior are not implemented by this walkthrough.

14 3 Permanent account to person association

Approved policy: Each login must link to exactly one registry person using a permanent internal person ID. Ambiguous matches require administrator review; names or editable contact emails must not automatically establish account ownership.

Current status and remaining work: Legacy browser name/email matching and staged protected-email fallbacks remain. Confirm existing links privately; a data repair must preserve attribution and resolve ambiguous cases explicitly.

14 4 Attribution does not confer editing rights

Approved policy: Creating a person record must not automatically grant permanent editing rights over that person. The linked person controls their profile; other editors require explicit permission or administrator authority. Preserve original creator attribution in audit history.

Current status and remaining work: Legacy canEditNode and staged ownership helpers use creator attribution. Their permission use must be separated from historical authorship without rewriting the original audit author.

14 5 Member editable and protected fields

Approved policy: Members may edit ordinary profile details and privacy preferences within their authorized scope. Administrator permissions, verified identity status, account-to-person links and recognized membership levels require an approved system process or administrator action. Existing approved progression processes, such as making the pledge, may update the appropriate level.

Current status and remaining work: The staged patch policy protects fields, but the active broad root write does not provide equivalent field-level enforcement. Approved progression workflows must be retained and checked individually.

14 6 Separate contact and login email

Approved policy: Contact email and login email remain separate. Changing contact email does not change login credentials or establish verified address ownership. Login email changes require a secure verification process. Neither email change alters the permanent person ID or automatically links or merges records.

Current status and remaining work: Existing email field names and verification semantics vary by workflow. The permanent link, credential-change verification and contact editing must be made consistent without inventing verification proof.

14 7 Sensitive cache lifecycle and transfer frugality

Approved policy: Clear the most sensitive browser data and close protected views on sign-out, account changes or detected revoked access. Balance security with minimizing unnecessary repeat downloads, particularly full-registry downloads. Preserve harmless display preferences and all shared registry records.

Current status and remaining work: The active full-graph localStorage cache remains. Sensitivity classes, account/permission scoping, selective loading, validity and revalidation remain to be designed and tested. No blanket cache purge or retention duration was approved.

14 8 Meaning of Open visibility

Approved policy: Open visibility means visible to authorized, signed-in IDRN users. Publication to the public internet requires a separate explicit choice; Open alone does not authorize public publication.

Current status and remaining work: This clarifies audience and consent. It is consistent with requiring an authorized login but does not repair the legacy graph field-access gap. No public-publication feature is created by this decision.

14 9 Separate verification claims and provenance

Approved policy: Keep email verification, person identity verification and organizational authority separate. For each claim record who or what confirmed it, when, and on what basis. Email verification alone must not automatically establish person identity, qualifications or organizational authority.

Current status and remaining work: A uniform claim/provenance model is not established across the system. Existing email fields and membership labels must not be relabeled as independently verified identity or organizational authority.

14 10 Verification history and permission review

Approved policy: When a verification claim expires, is withdrawn or is revoked, retain its history and reassess permissions that depend on it. Preserve past event participation, relationships and audit attribution. A change to verification does not itself authorize deletion of those records.

Current status and remaining work: Claim lifecycle states, dependent-permission review and timing require implementation. Historical events and attribution must remain distinct from present authority.

14 11 Scoped verification reviewers

Approved policy: Members may submit verification claims and supporting information. Confirmation requires an approved system verification process, a system administrator, or an explicitly appointed reviewer acting within a defined scope. Reviewer status does not grant administrator access or unrelated private-profile visibility.

Current status and remaining work: The current administrator/user roles do not establish this scoped reviewer feature. Reviewer appointment, scope enforcement and review auditing remain work; reviewer status must not confer unrelated private access.

14 12 Verification does not grant access

Approved policy: Email, identity or organizational verification alone must not automatically grant IDRN membership, registry access or administrator powers. IDRN approved onboarding and access rules determine permissions. A verified event contact may remain a nonmember.

Current status and remaining work: People, accounts, membership and event contacts already exist as different concepts. Exact onboarding and account-admission paths still need a uniform check that verification alone never grants authority.

14 13 Relationship assessment correction review

Approved policy: Members must not directly overwrite another person's relationship assessment or strength rating. Allow correction requests and administrator review; resolutions must preserve attribution and an audit trail.

Current status and remaining work: Existing edge-creator checks and private assessment endpoints provide partial related mechanisms, but no comprehensive correction-request/admin-review workflow is certified here. Preserve assessment provenance.

14 14 Account retirement preserves person history

Approved policy: Retiring or disabling a login removes its access while preserving the linked person permanent ID, registry record, relationships, event participation and audit history. Any record deletion or duplicate-person merge is a separate reviewed action.

Current status and remaining work: Account removal/disable operations and person archival are separate concerns. Each deployed endpoint needs review for access revocation, historical retention and cache/account-switch behavior; no account is retired now.

15 Remaining work and unverified areas

This is a description of IDRN's unfinished work, not an implementation authorization or external ecosystem assessment. These items follow from inspected code and the approved policies. They are not all new features: several complete or reconcile already staged work.

15.1 Reconcile authorization enforcement

The main registry's broad direct-client read/write grant is the largest policy mismatch. Closing it requires a compatible server view/edit path and corresponding database rules, together with checks on older callable endpoints. All editor paths, import, bulk tools, introductions, account management and feedback must be

considered. Enabling only the client flag, or changing only rules while old clients remain, is not a complete migration. Staged creator-ownership and Level 9 fullRead logic also need updates.

The new policy requires distinctions between standard editing, full visibility, administration and scoped claim review. The record does not yet define a final permission schema, who receives the new full-visibility grant or its review schedule. Administrator behavior must be made explicit rather than silently inherited from a historical level or browser flag.

15.2 Resolve identity and account continuity

Each login needs a stable, reviewed link to the intended person. Existing live accounts must be audited using appropriately authorized private evidence before a migration. A contact email or similar name cannot resolve an ambiguous link. Historical ID aliases, relationship endpoints, event references and authorship must remain coherent. Whether one person may have several login accounts or provider identities has not been decided in this walkthrough; no uniqueness migration should assume the answer.

Protected email fields, Authentication email verification and email-token proofs have different histories. A clean field value is not enough to certify a person or organizational role. The approved claim history/reviewer policies require explicit implementation and evidence. The report does not invent a new verification service or designate any external system as authoritative.

15.3 Balance caches and transfer cost

The owner explicitly asked to clear the most sensitive browser data on sign-out, account changes or detected revocation while minimizing repeat downloads. Shared records and harmless preferences remain intact. The present legacy cache does not meet that complete requirement. Classification and permission-aware cache validity must be determined before choosing retention or incremental loading. An offline device's previously copied information cannot be remotely recalled merely by revoking a server permission.

The compact overview/search/detail work is preserved, not active. Any replacement must retain authorized connection paths and search completeness. Production bandwidth, reconnect frequency, normalization writes and billed usage have not been measured sufficiently to claim a savings percentage. Events also needs indexing as it grows; its server-side scans are an explicitly documented initial compromise.

15.4 Complete lifecycle and operational evidence

Correction requests, scoped reviewers, claim expiry/revocation and separate visibility permissions need supported workflows and tests. Disabling login access must preserve the person and historical participation, relationships and attribution. Duplicate merging and record deletion remain separate reviewed actions.

Independent legacy function sources and separately hosted registration assets need targeted verification before broad claims about every onboarding, authorization and email path. Current account links, disabled/revoked behavior and profile flags should not be inferred from historical counts. The narrow health report does not replace billing, availability, backup or operational error monitoring. No fresh restore test or complete exposure assessment is claimed here.

16 Verification boundaries for future document revisions

A subsequent revision can move a claim from Approved policy to Implemented and release verified only when the relevant source, deployment and acceptance evidence exist. Code presence, a local flag, a passing pure unit test or a successful main merge alone is insufficient to establish all live security behavior.

The existing test discipline separates pure model tests, mocked browser transport, actual Firebase emulators, HTTP authentication boundaries, full interface behavior, live static verification and explicitly approved live data/mail tests. Relevant roles include signed-out, standard user, self, unrelated user, administrator, Level 9 without an explicit visibility grant, scoped reviewer, disabled/revoked account and ambiguous identity. New policy checks must cover direct database/API access as well as visible buttons.

Failures and uncertain commits must retain accurate status: a stored Events record with a delayed summary differs from an unconfirmed email delivery. Updating the technical document after a scoped release should name the exact source baseline, function package, rule/config change, tests performed and limits of the conclusion. Historical notes remain evidence; their older status must not override a newer completed record.

17 Recorded release timeline

Period	Recorded event	Architectural significance
October 6 and 7	Security migration testing and reviewed account-link maintenance	Explicit person associations added; broad legacy route was not replaced
October 7 and 8	Email-confirmed invitations and review editing developed across distinct deployed packages	Mailbox proof, membership, login and personal connection consent remain different workflows
October 20	Narrow invitation loader/confirmation and two-file website release; controlled inbox test	Corrected recipients and locked same-list recovery certified for the selected release
October 20 and 21	Clean GitHub checkout, PR #1 invitation reconciliation and PR #2 listener/offline release	Preserved source kept separate; normal Git workflow restored
October 21	Independent Hostinger auto-deployment identified and paused; protection and website-only publisher established	One active scoped website route replaced overlapping broad publication
October 21	Owner's local app push produced commit ab2ecd9 and a one-file live build marker	Demonstrated local app to GitHub to corrected Actions to live path
October 22 and 23	Events production package, PR #5 and isolated two-function deployment	Separate administrator UI/API with shared storage and real registry lookups
October 23	PR #6 Create controls and PR #7 real-data/organization wording	Narrow UI releases; backend, rules and records unchanged
October 23	Fourteen policy decisions approved for this report	Intended behavior recorded; no implementation or live change follows merely from approval

18 Glossary

Term	Meaning in this document
Authentication	Establishing the login identity used for a request
Authorization	Deciding what that identity may read or change

Term	Meaning in this document
Person record	Persistent registry entity, potentially without a login
Account profile	IDRN authorization/application record indexed by Firebase UID
Verification claim	An asserted fact with a confirmation basis, distinct from access permission
Designator	IDRN engagement/status classification; current Level 9 visibility coupling is a legacy behavior
Relationship assessment	A relational judgment or strength value, not identity verification
Projection	Derived list/summary data rebuilt from an authoritative record
Revision	Version used to detect intervening edits
Operation ID	Identifier used to recognize the same attempted mutation and prevent duplicate application within its retained history
Staged	Present as code or design but not enabled in the active production route
Release authority	The exact package/configuration and evidence that establish what was deployed

19 Source reference index

References below point to the inspected checkout at the baseline identified on the first page, unless explicitly labeled preserved evidence. Paths identify source locations; they do not confer repository access. The companion local evidence manifest records file hashes and the inventory capture time for reproducibility. No raw production records or credentials are included.

S01 AGENTS.md

Latest local checkout handoff; ignored operational record identifying live PRs, source boundaries and release evidence.

S02 ../global-connections/docs/CURRENT-STATE.md

Preserved candidate current-state record, plus RELEASE-MAP.md and DECISIONS.md; latest completion entries supersede older notes.

S03 README.md

Application functions, graph representation, imports, taxonomy, relationship score and maintenance behavior; historical prose checked against current code.

S04 docs/relational-networking-principles.md

Recorded product principles and privacy/consent direction.

S05 src/app.js

Current UI, login restoration, designator labels, editing/visibility helpers and legacy person matching.

S06 src/storage.js

Firebase SDK setup, listener ownership, local cache, saves, private assessment routing and opt-in secure mode.

S07 src/registryAccessClient.js

Staged in-memory server transport; also src/registryEditOperations.js for deliberate revision-based edits.

S08 functions/registrySecurity.js

Staged actor, visibility, opaque-reference, ownership and patch policy; functions/registryAccess.js supplies non-exported handlers.

S09 src/registryBaseline.js

Content comparison excluding graph-layout and array-order noise.

S10 src/taxonomy.js

Shared classifications; src/countryLinks.js and src/sampleData.js support reference data.

S11 docs/registry-security-migration.md

Staged status, tests, historical account-link review and maintenance limits.

S12 security/events/database.rules.acceptance-baseline.json

Rules-only copy of the production baseline used for Events acceptance; not a rules deployment input.

S13 docs/releases/data-cost-listener-20261020.md

Scoped listener/offline implementation, real-SDK acceptance, known extra normalization write and cost-evidence limits.

S14 docs/member-invitation-batches.md

Original email-confirmed batch workflow and private workflow paths; later editing behavior is superseded by S15.

S15 docs/releases/invitation-review-20261020.md

Selected deployed source authority, exact packages and controlled inbox result; release-prep/invitation-review-20261020.json pins package manifests.

S16 docs/events/PRODUCTION-RELEASE-20261022.md

Production Events scope, backend checks, initial limits and release order; final completion is in S01/E01.

S17 src/events/production-client.js

Production Firebase session transport; src/events/shared-client.js supplies API calls, stale-response protection and operation-ID retries.

S18 docs/events/FORM-LABELS-20261023.md

Latest form release; docs/events/CREATE-CONTROLS-20261023.md and local completion record provide responsive tests and receipts.

S19 scripts/website-publisher.cjs

Immutable plan and SFTP/public checks; scripts/website-publish-manifest.json and .github/workflows/deploy-hostinger.yml define exact website scope and workflow gates.

S20 functions/systemHealthFunctions.js

Hourly monitoring/notification wrapper; functions/systemHealth.js defines aggregate scope and explicit exclusions. Inventory establishes deployment presence, not current job health.

S21 docs/onboarding-improvements.md

Local/proposed onboarding changes; docs/gatherings-and-member-development.md records future gathering policy. Neither is a complete live-release certification.

S22 functions/events/service.mjs

Events authentication, aggregate mutation and projection; functions/events/http.mjs and functions/events-production/index.mjs define HTTP and production wrappers.

S23 src/events/model.js

Validation and event/participation model; schedule.js, catalog.js and admin.js implement time zones, category fields and forms.

S24 functions/events/directory.mjs

Private in-process registry directory projection, TTL and paging; host-index.mjs is not the chosen production query path.

S25 docs/releases/hostinger-protection-20261021.md

Reviewed origin protection and cache considerations; later purge/completion recorded in S01/S02 and publisher notes.

E01 .release-events-production-20261022/backend-verification.json

Ignored sanitized completion evidence dated 2026-10-23T05:03:24.244Z; functions-after.json supplies the preserved name/runtime inventory.

E02 .release-events-labels-20261023/website-receipt.json

Ignored verified PR #7 receipt, Actions run 39906756036; exactly two website files uploaded.

P01 confirmed-decisions.json

Local owner-confirmed 14-policy register accompanying this document work. Intended policies only.

Release links

Latest website label release: https://github.com/jmattwallace/idrn-registry/pull/7 and https://github.com/jmattwallace/idrn-registry/actions/runs/39906756036. These links may require repository permission. PR #5 records initial Events publication; PR #6 records the Create-controls fix. The current report does not create a new release, PR or deployment.

20 Preserved deployed function inventory

The following name/runtime inventory is taken from the preserved Events deployment check. It is dated evidence of deployment presence, not a statement that every endpoint was independently source-audited or retested for this report. Runtime differences are intentional evidence that root functions/package.json cannot define the entire deployed system.

Function name	Runtime
activateMemberImmediately	nodejs20
activatePledgedMembers	nodejs20
auditInvitationReadiness	nodejs20

Function name	Runtime
checkIdrnSystemHealth	nodejs22
checkMemberInvitationEmail	nodejs20
checkMemberInvitationEmailByAccessLink	nodejs20
checkSignedInInvitationEmail	nodejs22
completeMemberRegistration	nodejs22
completePasswordReset	nodejs20
confirmMemberInvitationBatch	nodejs22
createAccessAccount	nodejs20
createMemberInvitation	nodejs20
createMemberInvitationByAccessLink	nodejs20
eventsAdminApi	nodejs22
loadContributionSession	nodejs20
loadCurrentAccountIdentity	nodejs20
loadIdrnSystemHealth	nodejs22
loadMemberInvitation	nodejs20
loadMemberInvitationBatch	nodejs22
loadMemberInvitationProfile	nodejs20
loadMemberInviteAccess	nodejs20
loadPersonalConnectionRequest	nodejs22
loadPrivateRelationshipAssessments	nodejs22
loadPublicIntakeToken	nodejs20
loadSignedInInvitationProfile	nodejs22
projectEventsSummary	nodejs22
recordLogin	nodejs20
removeAccessAccount	nodejs20
requestContributionLink	nodejs20
requestInvitationEmailChangeCode	nodejs20
requestMemberInvitationBatch	nodejs22
requestMemberInviteAccessLink	nodejs20
requestPublicIntakeLink	nodejs20
respondToPersonalConnectionRequest	nodejs22

Function name	Runtime
savePrivateRelationshipAssessment	nodejs22
searchContributionMatches	nodejs20
sendAccessInvitationEmail	nodejs20
sendExistingNodeInvitation	nodejs20
sendIntroductionEmail	nodejs20
sendInvitationFeedbackNotifications	nodejs22
sendSignedInMemberInvitation	nodejs22
submitContributionDashboard	nodejs20
submitContributionFeedback	nodejs20
submitInvitationFeedback	nodejs22
submitPublicIntake	nodejs20

Document boundary: This report ends at the IDRN implementation and owner-approved requirements. Assessment of another ecosystem, future integration design, grants of access and live-system changes are outside its scope.